

MongoDB: criando coleções com restrições

Uma sequência do básico ao profissional

1. required + bsonType
2. enum + minimum/maximum
3. unique index
4. arrays e subdocumentos
5. pattern e validação completa

Roteiro

começar
simples
evoluir por
níveis
testar erros
comparar com
SQL

Por que começar simples?

- No relacional, você já conhece NOT NULL, tipo, UNIQUE e CHECK.
- No MongoDB, a ideia é parecida, mas os comandos ficam em lugares diferentes.
- Se começarmos por regex e arrays aninhados, a regra fica escondida dentro da sintaxe.
- A melhor aprendizagem é em camadas: uma restrição nova por vez.

Primeiro entender a regra. Depois sofisticar a validação.

A escada da aula

Nível 1

Obrigatório
required

Nível 2

Tipo de dado
bsonType

Nível 3

Lista permitida
enum

Nível 4

Intervalo
minimum /
maximum

Nível 5

Não repetir
unique index

Nível 6

Lista simples
array / minItems

Nível 7

Objeto interno
object

Nível 8

Lista de objetos
items

Nível 9

Formato
pattern

Nível 10

Projeto final
schema completo

Base relacional mínima para começar

```
CREATE TABLE alunos (  
  id_aluno SERIAL PRIMARY KEY,  
  nome VARCHAR(100) NOT NULL,  
  email VARCHAR(120) NOT NULL,  
  idade INT NOT NULL,  
  ativo BOOLEAN NOT NULL  
);
```

- Aqui ainda não temos regex, CHECK complexo ou relacionamento.
- O objetivo é traduzir o básico: campo obrigatório e tipo de dado.
- Depois que isso estiver claro, adicionamos novas restrições.

Nível 1: campos obrigatórios com required

```
db.createCollection("alunos", {  
  validator: {  
    $jsonSchema: {  
      bsonType: "object",  
      required: ["nome", "email", "idade", "ativo"]  
    }  
  }  
})
```

required é o equivalente mais direto do NOT NULL.

Nível 2: tipos com bsonType

```
db.createCollection("alunos", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["nome", "email", "idade", "ativo"],
      properties: {
        nome: { bsonType: "string" },
        email: { bsonType: "string" },
        idade: { bsonType: "int" },
        ativo: { bsonType: "bool" }
      }
    }
  }
})
```

Agora o MongoDB não aceita idade como texto nem ativo como número.

Nível 3: valores permitidos com enum

```
// Relacional  
CHECK (status IN ('ativo', 'inativo', 'trancado'))  
  
// MongoDB  
status: {  
  enum: ["ativo", "inativo", "trancado"]  
}
```

- enum limita o campo a uma lista fechada.
- É ótimo para status, tipo, categoria, situação e perfil.
- O valor precisa bater exatamente com uma opção da lista.

```
db.alunos.insertOne({ nome: "Lia", email: "lia@email.com", idade: 20, ativo: true, status: "pendente" })  
// deve falhar
```

Nível 4: intervalos com minimum e maximum

```
// Relacional
CHECK (idade >= 18)
CHECK (nota >= 0 AND nota <= 10)

// MongoDB
idade: { bsonType: "int", minimum: 18 },
nota: { bsonType: "double", minimum: 0, maximum: 10 }
```

Use minimum e maximum quando a regra é uma faixa numérica.

Nível 5: UNIQUE vira índice único

```
// Relacional  
email VARCHAR(120) UNIQUE NOT NULL  
  
// MongoDB  
// 1. required garante que o campo exista  
// 2. bsonType garante que seja string  
// 3. índice unique impede repetição  
  
db.alunos.createIndex({ email: 1 }, { unique: true })
```

Unicidade não é regra de JSON Schema; é regra de índice.

Nível 6: arrays simples

```
disciplinas: {  
  bsonType: "array",  
  minItems: 1  
}  
  
// Documento válido  
{  
  nome: "Ana",  
  email: "ana@email.com",  
  idade: 22,  
  ativo: true,  
  disciplinas: ["Banco de Dados", "NoSQL"]  
}
```

- Array representa uma lista dentro do documento.
- minItems exige uma quantidade mínima de itens.
- Neste nível ainda não validamos cada item da lista.

Nível 7: subdocumento simples

```
endereco: {  
  bsonType: "object",  
  required: ["cidade", "uf"],  
  properties: {  
    cidade: { bsonType: "string" },  
    uf: { bsonType: "string" }  
  }  
}
```

- Subdocumento é um objeto dentro do documento principal.
- Ele pode ter seus próprios campos obrigatórios.
- É útil para endereço, contato, perfil, configuração e medidas.

Só depois disso faz sentido validar CEP e UF com pattern.

Nível 8: arrays de objetos

```
matriculas: {  
  bsonType: "array",  
  items: {  
    bsonType: "object",  
    required: ["curso", "semestre", "situacao"],  
    properties: {  
      curso: { bsonType: "string" },  
      semestre: { bsonType: "int", minimum: 1 },  
      situacao: { enum: ["cursando", "aprovado", "reprovado"] }  
    }  
  }  
}
```

- items define a regra de cada item do array.
- Cada matrícula vira um objeto validado.
- Aqui aparecem validações internas mais profissionais.

Nível 9: formatos com pattern

```
// Só depois de entender required, bsonType e object
email: { bsonType: "string", pattern: "^.+@.+\\.+$" },
cpf: { bsonType: "string", pattern: "^[0-9]{11}$" },
cep: { bsonType: "string", pattern: "^[0-9]{8}$" },
uf: { bsonType: "string", pattern: "^[A-Z]{2}$" }
```

- pattern usa expressão regular.
- É poderoso, mas deve entrar no final da sequência.
- O aluno precisa entender a regra antes de decorar a regex.

Nível 10: coleção completa de alunos

```
db.createCollection("alunos", {
  validator: { $jsonSchema: {
    bsonType: "object",
    required: ["nome", "email", "idade", "ativo", "status"],
    properties: {
      nome: { bsonType: "string", minLength: 3 },
      email: { bsonType: "string" },
      idade: { bsonType: "int", minimum: 18 },
      ativo: { bsonType: "bool" },
      status: { enum: ["ativo", "inativo", "trancado"] },
      disciplinas: { bsonType: "array", minItems: 1 },
      endereco: { bsonType: "object", required: ["cidade", "uf"] }
    }
  }}
})
db.alunos.createIndex({ email: 1 }, { unique: true })
```

Projeto final: do relacional ao MongoDB

- Receber uma modelagem relacional com restrições.
- Escolher a coleção principal do documento.
- Decidir o que será incorporado e o que será referenciado.
- Criar a validação em níveis: básico, valor, unicidade, arrays e formato.
- Testar documento válido, documento incompleto, tipo errado e valor fora da regra.

A entrega deve mostrar a origem relacional de cada restrição MongoDB.

Resumo da correlação com o relacional

NOT NULL	required	Campo precisa existir
TIPO SQL	bsonType	string, int, bool, date
CHECK IN	enum	Lista de valores permitidos
CHECK numérico	minimum / maximum	Intervalo permitido
UNIQUE	createIndex unique	Não repetir entre documentos
FK	referência ou incorporação	Decisão de modelagem